

RBE 4701 - Project 2 Report

Team 05 - Brendan Bacon, Rowan Michaud, & Jake Claybrook

For the second project of RBE-4701: Artificial Intelligence in Robotics, we were tasked with coding and training an AI using Q-Learning to play the game bomberman. Our character was limited to two actions, move and place bombs. The rest of the methods we were provided helped us with analyzing the map, enemies, and most importantly the exit. The goal was for our AI to reach the exit on 50% or more of all survival variants we were provided.

We began with the same AI as we used in Project 1. Our AI works with the same 4 main states, those being HIDE, BOMB, EVADE, and SEEK_EXIT. To reiterate how these states work, the AI will enter the HIDE state if a bomb or explosion exists in the world. If not, it then checks if there is no path to the exit or if it has not made progress for some time while not currently being under threat by any monsters, and if so it enters the BOMB state. Next it checks if it is currently being threatened by any monsters, determined by comparing its current threat level to a threshold set, and if it is, the AI enters the EVADE state. If none of the conditions for these three states are met, the AI simply enters the SEEK_EXIT state. If the AI is in the SEEK_EXIT state, it simply follows the optimal path calculated with the current world state to reach the exit. If it is in the BOMB state, it drops a bomb. For both the EVADE and HIDE states, the AI runs a search algorithm to find the best move given the current world state.

Building off of this, we implemented Approximate Q-Learning. We assigned weights to many different aspects of the game to help assemble our Approximate Q-Value function. These aspects are defined as the following:

bias	The Bias constant required for a node of Q-Learning
progress	Is a Breadth First Search step gained towards the exit by taking this next move?
exit_disc	The distance to the exit of the level
mon_adj	Would you be next to a monster by taking this move?

mon_d2	Would you be within 2 squares of a monster by taking this move?
mon_d3	Would you be within 3 squares of a monster by taking this move?
mon_dist	Are you going to be entering a distance within 4 squares of a monster by taking this move?
ahead_adj	If a monster is moving in its current path, will taking this move put us next to a monster?
ahead_d2	If a monster is moving in its current path, will taking this move put us within 2 squares of a monster?
escape	A fraction of the cells you can go into our ESCAPE_DEPTH path without hitting a monster
in_blast	Will be active if you are standing within the cells of a bomb's blast.
blast_urg	A fraction based on how many moves you can make until a given bomb explodes.
blast_steps	Number of steps needed to exit a bomb's blast radius.
wait	The best course of action being
dies	Weighted as -10
wins	Weighted as +10

These weights are kept in two Json files called `q_weights_v1_4.json` and `q_weights_v5.json`. One of the most interesting things that we found after testing was the difference in weights for immediate progress, and those associated with danger. The progress weight is roughly +0.19 while `exit_disc` is -1.94. This shows that the learning places value on getting closer to the exit, but the distance based feature has a much larger coefficient. The actual contribution for each feature however, really depends on how often they are being used while running the AI.

Our AI was tested on 5 different variant situations. The AI was tested on our own variant maps so that testing would be different from Project 1. In the first variant, our AI is set in an empty world with no monsters. The first world consists of empty, traversable squares, destroyable walls, and out of bounds areas. Additionally, there is an exit square that will count as the goal to reach. Once our AI paths to the exit, the game ends and our AI has won. Our AI had no trouble with this variant.

```
project2/variant1: exit 500/500  dead  0  timeout  0  avg steps  80.0  avg score  4989.0
```

In the second variant, there is now a single “stupid” monster that will pick its next move at random every step. The start of the variant is the same as variant 1, with the AI following its A* path towards the exit. Once it nears the monster, though, it uses its Q-Learning evaluation to select actions that maintain a safe distance while still trying to get closer to the exit. The monster’s danger zone is wide enough that it isn’t safe to pass it while it is in the middle of the open space, but as it randomly walks it eventually goes toward one of the edges of the world, and the AI has an opportunity to slip by it. After it passes the monster, it then switches back to A* to finish out the path to the exit. Testing this variant we found our AI to have a 97% pass rate.

```
project2/variant2: exit 486/500  dead  9  timeout  5  avg steps  132.0  avg score  4691.9
```

In the third variant, the monster changes from being “stupid” to being “self preserving”. Like the previous monster, it will go straight until it reaches an obstacle, then move randomly among the available moving cells. However, if the available moving cell would be an explosion, the monster would move away from it so as to not die. This keeps it so that the monster will keep itself alive, keeping the obstacle you have to pass from easily being removed. In our testing, we found that our AI was able to survive around 91% of the time.

```
project2/variant3: exit 456/500  dead 42  timeout  2  avg steps  99.8  avg score  4129.1
```

The fourth variant makes another adjustment to the monster to make it “aggressive”, with it now able to sense and attack the AI from even further away, from 1 Chebyshev distance to 2. The start of the variant is the same yet again, and the AI uses its Q-Learning evaluation to maintain spacing from the monster just far enough away that the monster does not detect it. It does this until it finds an opportunity to slip past it to the exit like in previous iterations. Variant four, like the previous variant, also had

around a 91% pass rate.

```
project2/variant4: exit 456/500  dead 43  timeout 1  avg steps 143.6  avg score 4079.3
```

The fifth and final variant has a significant increase in difficulty, with it adding a “stupid” monster in addition to the “aggressive” one from variant 4. This makes it so that our AI will have to navigate two monsters to reach the exit, possibly having to deal with both at once. To pass, our AI had to find 2 opportunities to pass a monster to reach the exit, with the second of which being particularly dangerous due to it being stuck in between the monsters. If they move in just the wrong way, our AI could be stuck with no way to escape them. What we saw during testing was that our AI passed around 68% of the time for variant five. This is an important thing to note as it shows that our AI is not as consistent as we would like it to be.

```
project2/variant5: exit 340/500  dead 135  timeout 25  avg steps 185.1  avg score 1901.3
```

All of these variants use a different method for choosing actions than we used for the first project which we called “smart bomb” mode. This mode aims to have the AI only place bombs that break the next wall on the way to the exit. This was implemented after we noticed the AI getting stuck in bomb loops where it would place the bomb repeatedly and impair its own progress. In addition to this, variant 5 uses an addition to this system that lowers the threat radius of the monsters. This allowed for the AI to be more aggressive with bomb placements, which resulted in a small improvement in success rate.